# Two-Phase Commit: A Short Primer

*Supplementary reading for "Principles of Database Systems", Fall 2026*

## 1. The Atomic Commitment Problem

A distributed transaction updates data stored on several independent database nodes. Each node can commit or abort its local part, but the transaction as a whole must be atomic: either every participant commits, or every participant aborts. A node cannot simply commit when its own work succeeds, because another node may fail to apply its part. Atomic commitment protocols solve this problem, and two-phase commit (2PC) is the most widely used one.

## 2. Roles

One node acts as the **coordinator**; the nodes that executed parts of the transaction are the **participants**. The coordinator is often the node where the transaction started. Every node keeps a write-ahead log on stable storage so that it can recover its decisions after a crash.

## 3. Phase One: Prepare (Voting)

The coordinator sends a PREPARE message to every participant. A participant that can commit its part force-writes a *prepared* log record, which guarantees that it will be able to commit even after a crash, and replies YES. A participant that cannot commit writes an abort record and replies NO. After voting YES, a participant gives up its right to abort on its own: it is now in the **uncertain** (prepared) state and must wait for the coordinator's decision.

## 4. Phase Two: Commit or Abort (Decision)

If every participant voted YES, the coordinator force-writes a commit record and sends COMMIT to all participants; this log write is the moment the transaction is committed. If any participant voted NO or did not answer before a timeout, the coordinator decides to abort and sends ABORT. Each participant applies the decision, writes it to its log, releases its locks, and replies with an acknowledgment. When all acknowledgments have arrived, the coordinator writes an end record and can forget the transaction.

## 5. Handling Failures

If a participant crashes before voting, the coordinator times out and aborts the transaction. If a participant crashes after voting YES, on recovery it finds the prepared record in its log and asks the coordinator for the outcome. If the coordinator crashes before writing its decision, it aborts the transaction after recovery. Message loss is handled by timeouts and retransmission.

## 6. Why 2PC Is a Blocking Protocol

The weakness of 2PC appears when the coordinator crashes **after** the participants have voted YES but **before** they receive the decision. The prepared participants cannot commit, because some other participant might have voted NO, and they cannot abort, because the coordinator might already have decided to commit. They must keep holding their locks until the coordinator recovers. This is why 2PC is called a **blocking** protocol: a single coordinator failure can block the other nodes, and the data they have locked stays unavailable.

## 7. Beyond 2PC

Three-phase commit (3PC) adds a pre-commit phase to avoid blocking under the assumption of a synchronous network with bounded delays, an assumption that rarely holds in practice. Modern systems instead replicate the coordinator's state with a consensus protocol such as Paxos or Raft, so that a new coordinator can take over and finish the protocol when the old one fails.